

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по ИДЗ
по дисциплине «Распределённые алгоритмы»
Тема: Реализация алгоритма умножения матриц на Promela
и верификация в SPIN

Студент гр. 3343	_____	Малиновский А. А.
Студент гр. 3342	_____	Романов Е. А
Студент гр. 3342	_____	Галеев А. Д.
Преподаватель	_____	Шошмина И. В.

Санкт-Петербург

2026

Задание.

Вариант ИДЗ 2.4.

Разработайте параллельный алгоритм умножения двух матриц $n \times n$ для P процессов так, чтобы параллельно вычислялись столбцы результирующей матрицы. Исходные матрицы загружены на процессы заранее. Сформулировать свойства алгоритма на языке LTL. Обмен данными, если он необходим, осуществляется с помощью **каналов**. По завершении умножения 0-й процесс выводит результат.

Выполнение работы.

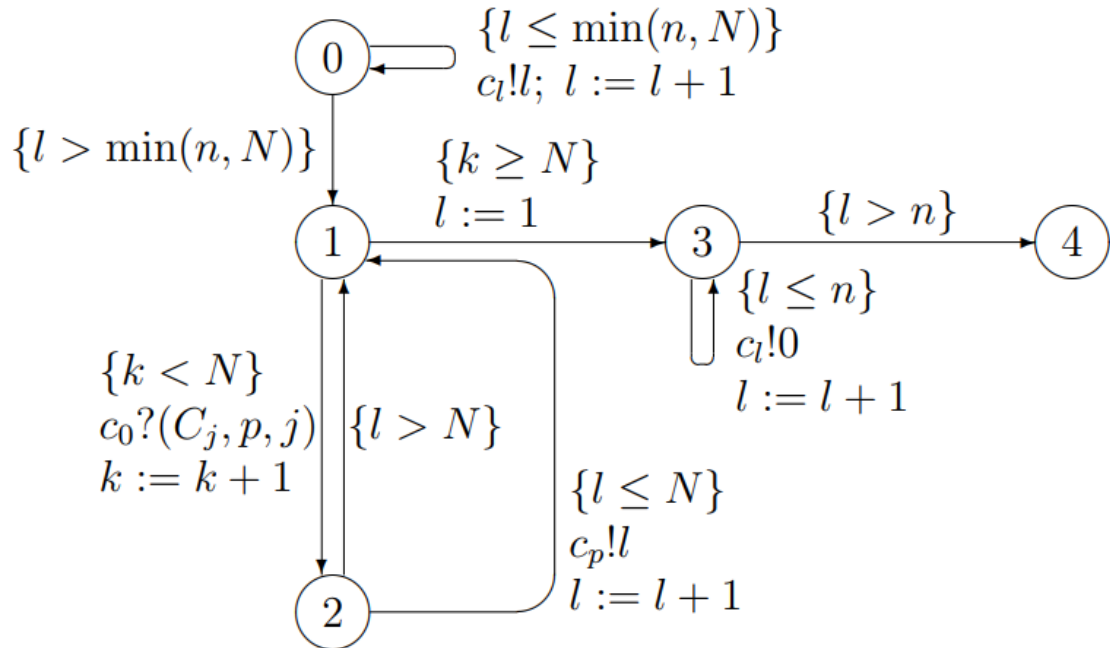
Наша задача состоит в написании распределённой программы, вычисляющей произведение матриц A и B . РП P имеет вид $\{P_0, \dots, P_n\}$, где $n \geq 1$. Процесс P_0 называется менеджером, процессы $P_1, \dots, P_n$ называются работниками.

Неформально работу излагаемой в этом пункте РП P для перемножения матриц A и B можно описать следующим образом. Каждый работник имеет матрицы A и B . Работа менеджера заключается в назначении задач работникам и приеме результатов от них. Каждая задача работнику заключается в вычислении одного столбца матрицы $C = AB$. Менеджер назначает эту задачу путем послыки работнику номера i того столбца матрицы B , на который работник должен умножить A . Когда менеджер получает от работника результат (т.е. сообщение с вычисленным столбцом $AB_j = C_j$), он посылает этому работнику новый номер (если еще есть неназначенные строки) или 0 (если неназначенных строк нет).

Спецификация изложенной выше РП перемножения матриц представляет собой следующее утверждение: после завершения выполнения этой РП верно равенство $C = AB$, которое эквивалентно утверждению $\forall j = 1, \dots, N \ C_j = AB_j$.

Далее мы определяем РП $P = \{P0, P1, \dots, Pn\}$, перемножения матриц. Входными данными P являются A, B (матрицы-сомножители), N (число строк в A), n (число работников).

В процессе $P0$ столбцы матрицы-результата C записываются в переменные $C1, \dots, CN$. Процесс $P0$ имеет следующий вид:



$$Pre(P_0) = (N \geq 1) \wedge (l = 1) \wedge (k = 0).$$

Рисунок 1 – процесс $P0$

Поясним работу процесса $P0$:

- переход $0 \rightarrow 0$: менеджер $P0$ дает первые задания работникам: посылает работникам по очереди номер l (нумерация от единицы) столбца матрицы B до тех пор, пока либо не дойдёт до последнего столбца, либо все имеющиеся работники не получат задачи,
- переход $0 \rightarrow 1$: вход в цикл (состоящий из перехода $1 \rightarrow 2$ и двух переходов $2 \rightarrow 1$) приема результатов от работников и назначения им новых заданий: – переход $1 \rightarrow 2$: получение от работника результата выполнения полученной им задачи. Значение k – количество принятых результатов. Переходы $2 \rightarrow 1$: выдача работнику новой задачи (если такая задача имеется) либо пустой переход (если новых задач

нет),

- переход $1 \rightarrow 3$: завершение приема результатов от работников, вход в цикл $3 \rightarrow 3$ посылки работникам сигнала окончания работы (0),
- переход $3 \rightarrow 4$: завершение работы

Процесс P_i для $i > 0$ (работник) имеет следующий вид:

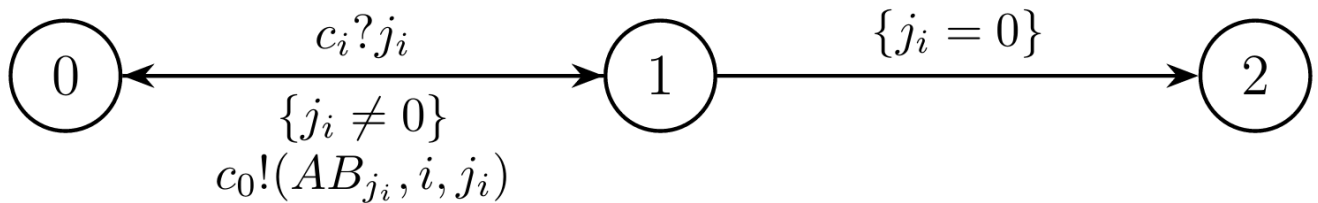


Рисунок 2 – процесс работник

Символ i (рис. 2) обозначает константу, равную номеру этого процесса.

Поясним работу процесса P_i :

- переход $0 \rightarrow 1$: P_i получает от менеджера число j_i , которое интерпретируется либо как номер столбца матрицы B , на который нужно умножить A и послать вычисленное произведение менеджеру (если $j_i \neq 0$), либо как сигнал об окончании работы (если $j_i = 0$),
- переход $1 \rightarrow 0$: посылка менеджеру вычисленного произведения,
- переход $1 \rightarrow 2$: окончание работы.

Запуск программы:

Запустим программу с матрицами A и B (см. Рис. 3), количество потоков поставим $P=4$.

$$A = \begin{pmatrix} 1 & 2 & 3 & 1 \\ 4 & 5 & 6 & 1 \\ 7 & 8 & 9 & 1 \\ 1 & 1 & 1 & 2 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 1 \\ 1 & 2 & 3 & 1 \end{pmatrix}$$

Рисунок 3 – исходные матрицы

Запускаем программу:

Через gui:

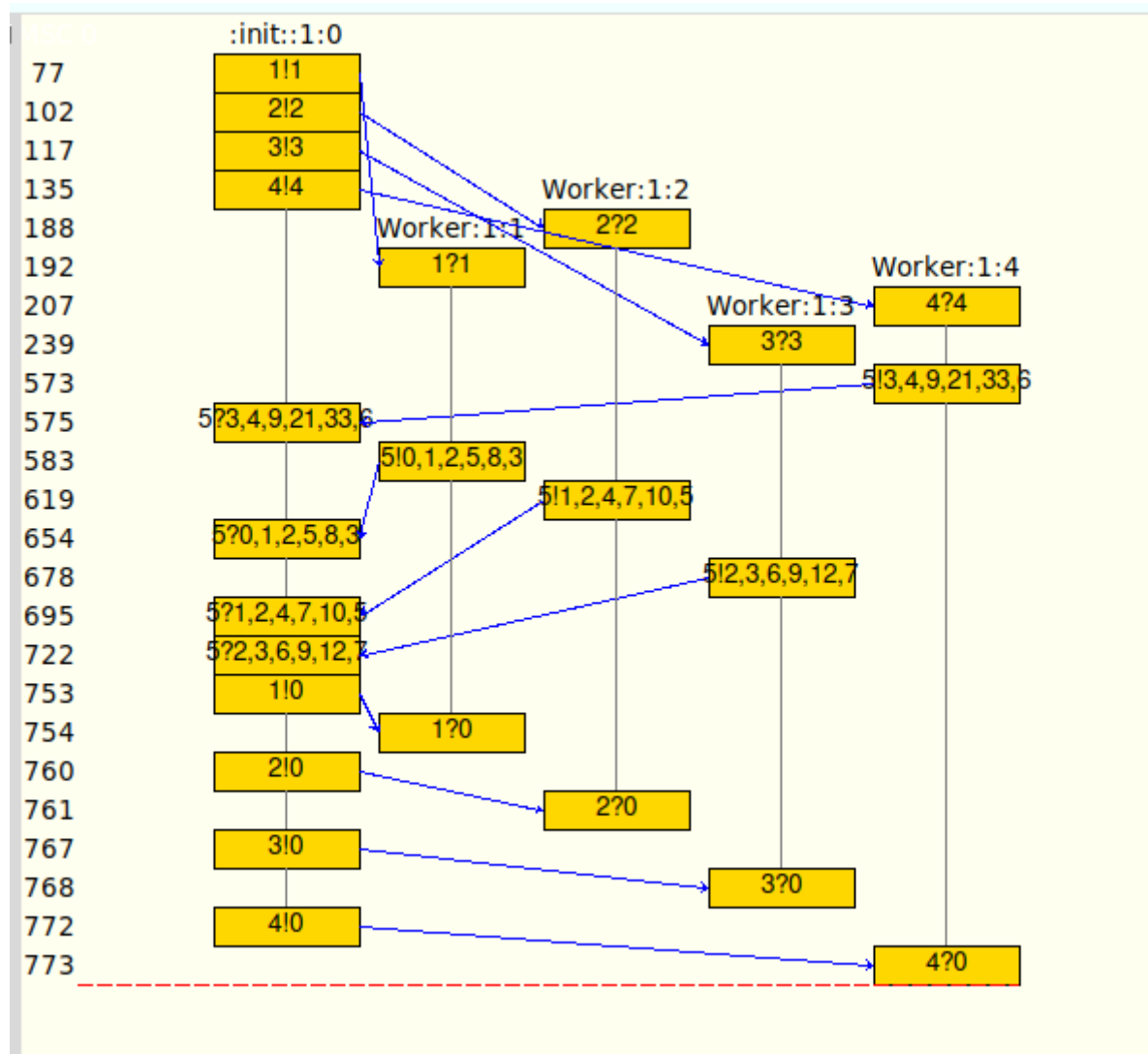


Рисунок 4 – результат работы программы

Через консоль:

```
immortal@Huawei:~/spin/Distributed-Algorithms/matmul$ spin matmul_custom.pml
ltl safety_bounded: [] ((k<=4))
ltl done_after_all: [] ((! ((done==1))) || ((k==4)))
ltl cols_stable: [] ((! ((k==4))) || ([] ((k==4))))
ltl liveness_done: <> ((done==1))
ltl progress_collect: [] ((! ((k<4))) || (<> ((k==4))))
ltl cols_then_done: [] ((! ((k==4))) || (<> ((done==1))))
  Matrix A:
    1      2      3      1
    4      5      6      1
    7      8      9      1
    1      1      1      2
  Matrix B:
    1      0      0      1
    0      1      0      2
    0      0      1      1
    1      2      3      1
  Result matrix C = A * B:
    2      4      6      9
    5      7      9      21
    8     10     12     33
    3      5      7      6
5 processes created
immortal@Huawei:~/spin/Distributed-Algorithms/matmul$
```

Рисунок 5 – результат работы программы

Верификация программы.

Для верификации программы были составлены 6 свойств на языке LTL:

1) Безопасность (safety)

$\text{ltl safety_bounded } \{ [] (k \leq N) \}$

Счётчик принятых столбцов никогда не превышает N. Менеджер ждёт ровно N.

$\text{ltl done_after_all } \{ [] (done == 1 \rightarrow k == N) \}$

done = 1 только после сборки всех N столбцов. Свойство гарантирует, что не произойдёт преждевременного завершения.

$\text{ltl cols_stable } \{ [] (k == N \rightarrow [] (k == N)) \}$

k монотонен и достигнув N, не убывает.

2) Живость (liveness)

$\text{ltl liveness_done } \{ <> (done == 1) \}$

Алгоритм всегда завершается. Нарушение означало бы взаимную блокировку или голодание.

$\text{ltl progress_collect } \{ [] (k < N \rightarrow <> (k == N)) \}$

Прогресс гарантирован из любого промежуточного состояния, не только из начального. То есть даже при частично собранном C оставшиеся столбцы обязательно будут подсчитаны.

$\text{ltl cols_then_done } \{ [] (k == N \rightarrow <> (done == 1)) \}$

Как только все столбцы собраны, менеджер обязательно дойдёт до done, то есть программа завершит работу.

Проверим наличие ошибок через консоль:

```
immortal@Huawei:~/spin/Distributed-Algorithms/matmul$ spin -a matmul.pml && gcc -O2 -o pan pan.c
for p in safety_bounded done_after_all cols_stable \
    liveness_done progress_collect cols_then_done; do
    echo "=== $p ==="
    ./pan -a -f -N $p | grep -E "errors:|completed|violated"
done
ltl safety_bounded: [] ((k<=4))
ltl done_after_all: [] ((! ((done==1))) || ((k==4)))
ltl cols_stable: [] ((! ((k==4))) || ([ ((k==4))))
ltl liveness_done: <> ((done==1))
ltl progress_collect: [] ((! ((k<4))) || (<> ((k==4))))
ltl cols_then_done: [] ((! ((k==4))) || (<> ((done==1))))
the model contains 6 never claims: cols_then_done, progress_collect, liveness_done, cols_stable, done_after_all, safety_bounded
only one claim is used in a verification run
choose which one with ./pan -a -N name (defaults to -N safety_bounded)
or use e.g.: spin -search -ltl safety_bounded matmul.pml
=== safety_bounded ===
State-vector 360 byte, depth reached 1123, errors: 0
=== done_after_all ===
State-vector 360 byte, depth reached 1123, errors: 0
=== cols_stable ===
State-vector 360 byte, depth reached 1123, errors: 0
=== liveness_done ===
State-vector 360 byte, depth reached 1120, errors: 0
=== progress_collect ===
State-vector 360 byte, depth reached 1123, errors: 0
=== cols_then_done ===
State-vector 360 byte, depth reached 1123, errors: 0
immortal@Huawei:~/spin/Distributed-Algorithms/matmul$
```

Рисунок 6 – результат работы программы

Сравнение с вариантом 2.3

Вариант 2.3 предполагает обмен через разделяемые переменные. Решение может иметь такой вид:

Также инициализируются 3 матрицы: А, В – исходные и С – для хранения результата. Заводятся 2 массива:

- `byte task_col[P]` – массив размера P (к-во процессов), из которого процессы будут брать номер столбца, который они должны вычислить
- `bit task_ready[P]` – массив размера P, в который процессы-работники будут отправлять «сигнал» о том, что задача выполнена. Менеджер пишет 1 в ячейку процесса, если есть ещё задачи. Работник пишет 0 в свою ячейку, когда с прошлой задачей он уже справился и готов взять следующую.

После того как процесс-менеджер заполнил массив с задачами и с сигналами, он ждёт, пока количество записанных от процессов-работников сигналов не будет равно N и пока весь массив `task_ready` не будет заполнен сигналами 0. Тогда получится ситуация, когда все результаты будут подсчитаны.

При такой архитектуре блокировки не понадобятся, потому что работники будут писать только в свои столбы матрицы С и не пересекутся друг с другом. А при работе с массивами `task_ready` и `task_col` получится, что:

`task_col` – работник только читает, менеджер только пишет

`task_ready` – работник только пишет, менеджер только читает.

Вывод

Разработан параллельный алгоритм умножения двух матриц $n \times n$ для P процессов так, чтобы параллельно вычислялись столбцы результирующей матрицы. Сформулированы свойства алгоритма на языке LTL. Алгоритм работает корректно, все LTL свойства выполняются без ошибок.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

matmul.pml

```
#define N      4          /* размер матриц N x N          */
#define P      4          /* число работников      */
#define STOP 0          /* "конец работы" в канале task */

/* Тип "столбец" — массив длины N в одном сообщении */
typedef Col { byte v[N] };

chan task[P] = [1] of { byte };
chan result = [P] of { byte, byte, Col };
bit done = 0;
byte k = 0;          /* число принятых менеджером столбцов-
результатов */

/* ---- Печать матрицы N x N (inline-макро) ---- */
inline print_matrix(M)
{
    byte pi = 0;
    do
        :: (pi < N) ->
            byte pj = 0;
            do
                :: (pj < N) -> printf("%d ", M[pi*N + pj]); pj++
                :: (pj == N) -> break
            od;
            printf("\n");
            pi++
        :: (pi == N) -> break
    od
}

inline init_AB(A, B)
{
    A[0*N + 0] = 1;
    A[0*N + 1] = 2;
    A[0*N + 2] = 3;
    A[0*N + 3] = 1;
    A[1*N + 0] = 4;
    A[1*N + 1] = 5;
    A[1*N + 2] = 6;
    A[1*N + 3] = 1;
    A[2*N + 0] = 7;
    A[2*N + 1] = 8;
    A[2*N + 2] = 9;
    A[2*N + 3] = 1;
    A[3*N + 0] = 1;
    A[3*N + 1] = 1;
    A[3*N + 2] = 1;
    A[3*N + 3] = 2;
```

```

B[0*N + 0] = 1;
B[0*N + 1] = 0;
B[0*N + 2] = 0;
B[0*N + 3] = 1;
B[1*N + 0] = 0;
B[1*N + 1] = 1;
B[1*N + 2] = 0;
B[1*N + 3] = 2;
B[2*N + 0] = 0;
B[2*N + 1] = 0;
B[2*N + 2] = 1;
B[2*N + 3] = 1;
B[3*N + 0] = 1;
B[3*N + 1] = 2;
B[3*N + 2] = 3;
B[3*N + 3] = 1;
}

proctype Worker(byte id)
{
    /* СВОИ копии A и B – никакой shared memory */
    byte A[N*N];
    byte B[N*N];
    byte column_number, i, j, s;
    Col col;

    init_AB(A, B);

    /* Цикл "получить задачу – выполнить – вернуть результат" */
    do
        :: task[id] ? column_number ->
            if
                :: (column_number == STOP) -> break
                состояние 2: конец */
            :: else ->
                /* column_number ∈ {1..N}: задача */
                /* col[i] = sum_k A[i,k] * B[k, column_number-1]
                (column_number 1-индексное) */
                i = 0;
                do
                    :: (i < N) ->
                        s = 0;
                        j = 0;
                        do
                            :: (j < N) ->
                                s = s + A[i*N + j] * B[j*N + (column_number -
1)];
                                j++;
                            :: (j == N) -> break
                        od;
                        col.v[i] = s;
                        i++;

```

```

        :: (i == N) -> break
    od;
    result ! id, column_number, col
fi
od
}

init {
    /* Локальные копии A и B — нужны только для печати (нет shared)
    */
    byte A[N*N];
    byte B[N*N];
    byte C[N*N];

    /* Переменные алгоритма (как в книге) */
    byte l;          /* следующий назначаемый номер столбца,
1..N              */
    byte rid;        /* id работника, приславшего
результат         */
    byte rj;         /* номер столбца, который он
считал             */
    Col col;

    init_AB(A, B);

    /* Запуск P работников */
    atomic {
        byte w = 0;
        do
            :: (w < P) -> run Worker(w); w++
            :: (w == P) -> break
        od
    }

    /* ---- Состояние 0 -> 0: первичная раздача задач ----
        Посылаем работникам по очереди номера столбцов 1..min(P, N).
    */
    l = 1;
    do
        :: (l <= P) && (l <= N) -> task[l - 1] ! l; l++
        :: else -> break
    od;
    /* теперь l = min(P, N) + 1 */

    /* ---- Состояния 1 -> 2 -> 1: приём результатов и довыдача задач
    ----
        Принимаем ровно N результатов; после каждого — выдаём новую
        задачу тому же работнику, если ещё есть, иначе пропускаем. */
    k = 0;
    byte i;
    do
        :: (k < N) ->
            result ? rid, rj, col;

```

```

/* Раскладываем столбец col в (rj-1)-й столбец C */
i = 0;
do
  :: (i < N) -> C[i*N + (rj - 1)] = col.v[i]; i++
  :: (i == N) -> break
od;
k++;
if
  :: (l <= N) -> task[rid] ! l; l++
  :: else      -> skip
fi
:: (k == N) -> break
od;

/* ---- Состояние 3 -> 3: рассылка сигнала окончания работы ----
  STOP отправляется ВСЕМ P работникам, включая тех, кто никогда
  не получал реальной задачи (случай P > N). */
byte q = 0;
do
  :: (q < P) -> task[q] ! STOP; q++
  :: (q == P) -> break
od;

/* ---- Печать A, B, C ---- */
printf("Matrix A:\n");
print_matrix(A);
printf("Matrix B:\n");
print_matrix(B);
printf("Result matrix C = A * B:\n");
print_matrix(C);

done = 1
}

ltl safety_bounded    { [] (k <= N) }
ltl done_after_all    { [] (done == 1 -> k == N) }
ltl cols_stable       { [] (k == N -> [] (k == N)) }
ltl liveness_done     { <> (done == 1) }
ltl progress_collect  { [] (k < N -> <> (k == N)) }
ltl cols_then_done    { [] (k == N -> <> (done == 1)) }

```